Setup

```
$ git clone https://github.com/mel-quark/zoo.git
$ cd zoo
$ bundle install
$ rails generate rspec:install
```

Models

interface to the database

create a model

`rails generate model model_name`

```
$ rails g model tiger
      invoke  active_record
      create    db/migrate/20150416231755_create_tigers.rb
      create    app/models/tiger.rb
      invoke    rspec
      create      spec/models/tiger_spec.rb
```

migration

```
class CreateTigers < ActiveRecord::Migration
  def change
    create_table :tigers do |t|

      t.timestamps null: false
    end
  end
end
```

db/migrate/20150416165107_create_tigers.rb

add an attribute

```
class CreateTigers < ActiveRecord::Migration
  def change
    create_table :tigers do |t|
     t.integer :appetite

      t.timestamps null: false
    end
  end
end
```

db/migrate/20150416165107_create_tigers.rb

or, we could...

destroy the model, and use the generate utility again.
this time: with columns.

destroy a model

`rails destroy model model_name`

```
      create      spec/models/tiger_spec.rb
$ rails d model tiger
      invoke  active_record
      remove    db/migrate/20150416231755_create_tigers.rb
      remove    app/models/tiger.rb
      invoke    rspec
      remove      spec/models/tiger_spec.rb
```

create a model

```
rails generate model model_name field:type field:type
```

```
      remove      spec/models/tiger_spec.rb
$ rails g model tiger appetite:integer
      invoke  active_record
      create    db/migrate/20150416232120_create_tigers.rb
      create    app/models/tiger.rb
      invoke    rspec
      create      spec/models/tiger_spec.rb
```

now, look at the migration file

migrate

```
rake db:migrate
```

run all pending migrations

how does it know?

```
$ sqlite3 db/development.sqlite3

sqlite> .tables
schema_migrations  tigers

sqlite> select * from schema_migrations;
20150416165107

sqlite>
```

now try...

```
sqlite> .exit

$ rake db:rollback
$ sqlite3 db/development.sqlite3

sqlite> .tables
schema_migrations  tigers

sqlite> select * from schema_migrations;
sqlite>
```

what's going on?

ORM

object relational mapping

Active Record

The default ORM for rails.

alternatives:
https://www.ruby-toolbox.com/categories/orm

Manual Creation

Create the table in the database (migration or in the DB shell directly), then create a file in app/models.

```
class Tiger < ActiveRecord::Base
end
```

For this to work, you must follow the model naming conventions.

Naming

Singular of the table name.

Model / Class	Table / Schema
`Article`	`articles`
`LineItem`	`line_items`
`Deer`	`deers`
`Mouse`	`mice`
`Person`	`people`

http://guides.rubyonrails.org/active_record_basics.html

Attributes

the columns in the table

```
ActiveRecord::Schema.define(version: 20150113071109) do

  create_table "topics", force: :cascade do |t|
    t.string   "title"
    t.text     "description"
    t.datetime "created_at",  null: false
    t.datetime "updated_at",  null: false
  end

  create_table "votes", force: :cascade do |t|
    t.integer  "topic_id"
    t.datetime "created_at", null: false
    t.datetime "updated_at", null: false
  end

end
"db/schema.rb" 29L, 1155C
```

Over-rides

Why?

Legacy database with non-descript names.

Table Name

```
class Topic < ActiveRecord::Base
  self.table_name = "input"
end
```

Column Name

```
class Topic < ActiveRecord::Base
  def description # alias
    self.title    # name in db table
  end
end
```

(alias)

rails console

interactive shell with your dev environment loaded

new

```
user = User.new do |u|
  u.name = "David"
  u.occupation = "Code Artist"
end
```

http://guides.rubyonrails.org/active_record_basics.html

save

```
user = User.find_by(name: 'David')
user.name = 'Dave'
user.save
```

http://guides.rubyonrails.org/active_record_basics.html

create

```
user = User.create(name: "David", occupation: "Code Artist")
```

http://guides.rubyonrails.org/active_record_basics.html

destroy

```
user = User.find_by(name: 'David')
user.destroy
```

http://guides.rubyonrails.org/active_record_basics.html

Queries

all

```
Tiger.all
```

http://guides.rubyonrails.org/active_record_basics.html

first

```
Tiger.first
```

http://guides.rubyonrails.org/active_record_basics.html

sample

`Tiger.all.sample`

find_by

```
user = User.find_by(name: 'David')
user.name = 'Dave'
user.save
```

http://guides.rubyonrails.org/active_record_basics.html

where

```
# find all users named David who are Code Artists and sort by
created_at in reverse chronological order
users = User.where(name: 'David', occupation: 'Code
Artist').order('created_at DESC')
```

http://guides.rubyonrails.org/active_record_basics.html

order

```
# find all users named David who are Code Artists and sort by
created_at in reverse chronological order
users = User.where(name: 'David', occupation: 'Code
Artist').order('created_at DESC')
```

http://guides.rubyonrails.org/active_record_basics.html

Editing

update

```
user = User.find_by(name: 'David')
user.update(name: 'Dave')
```

http://guides.rubyonrails.org/active_record_basics.html

update_all

```
User.update_all "max_login_attempts = 3, must_change_password =
'true'"
```

http://guides.rubyonrails.org/active_record_basics.html

Non DB Models

```
class Rando < ActiveRecord::Base
  attr_accessor :tiger

  def self.tiger
    Tiger.all.sample
  end
end
```

Put the in app/models and make sure the filename matches the class name.

rspec

test runner and assertion library

rspec-rails

TDD/BDD library that makes
rspec functionality available in rails.

run the tests

```
$ rake spec
```

```
Pending: (Failures listed here are expected and do not affect your suite's status)

  1) Tiger add some examples to (or delete) /Users/mel/work/UW/zoo/spec/models/tiger_spec.rb
     # Not yet implemented
     # ./spec/models/tiger_spec.rb:4


Finished in 0.00047 seconds (files took 1.64 seconds to load)
1 example, 0 failures, 1 pending
```

test structure

describe

test block. this example is for unit testing a model, specifically

```
require 'rails_helper'

RSpec.describe Tiger, type: :model do

end
```

describes can be nested

```
RSpec.describe Tiger, type: :model do

  describe "in the zoo" do
    it "doesn't do anything yet"
  end

end
```

```
*

Pending: (Failures listed here are expected and do not affect your suite's status)

  1) Tiger in the zoo doesn't do anything yet
     # Not yet implemented
     # ./spec/models/tiger_spec.rb:6


Finished in 0.00053 seconds (files took 1.61 seconds to load)
1 example, 0 failures, 1 pending
```

it

individual test

subject

reference to the implicit model being tested

```
RSpec.describe Tiger, type: :model do

  it "needs food" do
    expect(subject).to have_attribute(:appetite)
  end

end
```

pending tests

it call without a block
or calls "pending" explicitly
or prepend an it with an x (to make "xit")

assertion library

provides methods for comparing
expected values to results

http://www.rubydoc.info/gems/rspec-expectations